

DESCRIPCIÓN ESTRUCTURA 3 (QWEN 3.6)

Aquí tienes una guía completa, práctica y alineada con la convención oficial de OpenCode. He estructurado la respuesta para que puedas copiar, pegar y usar inmediatamente.

Función de cada archivo en `.ai/`

Archivo	Función principal	¿Qué define?	Nivel de prioridad
<code>system.md</code>	Prompt de sistema base	Identidad, tono, flujo de trabajo, formato de salida y comportamiento global del agente	 Alta (núcleo del agente)
<code>rules.md</code>	Reglas innegociables	Límites duros, seguridad, estilo de código, políticas de Git, restricciones de alcance	 Alta (filtro de validación)
<code>context.md</code>	Memoria técnica del proyecto	Stack, arquitectura, estructura de carpetas, esquema de BD, decisiones ya tomadas, endpoints	 Media (contexto operativo)
<code>agents.md</code>	Orquestación multi-agente	Definición de sub-roles, triggers, capacidades especializadas y rutas de delegación	 Baja/Media (solo si usas multi-agente)

Cómo interactúan en tiempo de ejecución

OpenCode no ejecuta estos archivos por separado. Los ensambla en un único prompt de sistema antes de cada interacción. El flujo interno es:

1. Carga: Lee los 4 archivos en el orden: `rules.md` → `context.md` → `agents.md` → `system.md`
2. Concatenación: Los fusiona en una única instrucción de sistema que se inyecta en el `role: "system"` del modelo.
3. Resolución de conflictos: Si hay contradicciones, las instrucciones que aparecen después suelen tener mayor peso (por proximidad al prompt del usuario), pero lo ideal es mantener coherencia absoluta.
4. Ejecución: El LLM responde aplicando la personalidad de `system.md`, respetando `rules.md`, usando `context.md` como referencia técnica y, si aplica, delegando según `agents.md`.

💡 Regla de oro: `system.md` debe ser el "manual de comportamiento", `rules.md` el "código penal", `context.md` la "documentación técnica" y `agents.md` el "organigrama".



Ejemplos reales y coherentes (listos para copiar)

♦ `.ai/system.md`

markdown

Rol

Eres un ingeniero de software senior especializado en el stack de este proyecto. Tu objetivo es entregar código limpio, mantenible y alineado a las convenciones del equipo.

Comportamiento

- Sé conciso pero completo. Evita rodeos.
- Si falta información crítica, haz 1-2 preguntas concretas antes de asumir.
- Prioriza legibilidad y pruebas sobre optimización prematura.

Flujo de trabajo

1. Analizar archivos relevantes y contexto
2. Proponer plan breve (máx. 3 pasos)
3. Implementar cambios atómicos
4. Sugerir comando de validación (lint/test)

Formato de salida

- Usa bloques de código con lenguaje especificado
- Separa lógica, UI y tests en archivos distintos

- Incluye comentarios solo para decisiones no obvias

♦ `.ai/rules.md`

markdown

Reglas obligatorias

- NUNCA modifiques archivos fuera de ``src/`` o ``prisma/`` sin confirmación explícita.
- NUNCA uses ``any`` en TypeScript. Usa tipos explícitos o ``unknown`` con validación.
- NUNCA expongas variables de entorno en el frontend o en logs.
- NUNCA omitas manejo de errores en llamadas a API, BD o servicios externos.
- SIEMPRE ejecuta ``npm run lint && npm run test`` antes de sugerir un commit.

Políticas de Git

- Commits atómicos y descriptivos (formato: ``type(scope): descripción``)
- No mezcles cambios de funcionalidad con refactor o estilos
- Si introduces breaking changes, documenta en ``CHANGELOG.md``

♦ `.ai/context.md`

markdown

Stack técnico

- Frontend: Next.js 14 (App Router), React 18, TypeScript 5.4, TailwindCSS
- Backend: tRPC, Prisma ORM, PostgreSQL 15
- Auth: NextAuth v5, JWT
- Deploy: Vercel, GitHub Actions (CI/CD)

Estructura clave

/src

- /app → Rutas y layouts (App Router)
- /components → Componentes reutilizables
- /lib → Utilidades, clientes de API, configs
- /prisma → Esquema y migraciones
- /server → Lógica de negocio y tRPC routers

Decisiones técnicas ya tomadas

- Usar Server Components por defecto. Client Components solo con ``"use client"``
- Validación de formularios con ``zod`` + ``react-hook-form``
- Estado global solo para UI (toast, modal). Datos de servidor via tRPC/React Query

♦ `.ai/agents.md`

markdown

```
# Agentes especializados
## frontend-dev
- Trigger: "UI", "component", "tailwind", "client"
- Capaz de: Generar componentes accesibles, responsive y con tipos estrictos
- Límite: No modifica lógica de servidor ni esquemas de BD

## backend-dev
- Trigger: "API", "prisma", "trpc", "DB", "auth"
- Capaz de: Crear routers, migraciones, validaciones, manejo de sesiones
- Límite: No toca componentes UI ni estilos

## qa-reviewer
- Trigger: "test", "review", "lint", "edge case"
- Capaz de: Sugerir pruebas unitarias/e2e, detectar code smells, validar cobertura
- Límite: No implementa funcionalidad nueva, solo valida y propone fixes
```

¿Falta algún archivo? (Recomendaciones 2026)

Tu estructura es sólida, pero según prácticas actuales de agentes de IA, estos 3 archivos opcionales pueden multiplicar tu productividad:

Archivo	Cuándo añadirlo	Qué resuelve
<code>tools.md</code>	Si el agente puede ejecutar comandos	Define qué herramientas tiene permitidas (<code>npm run</code> , <code>prisma migrate</code> , <code>git</code> , <code>docker</code> , etc.) y cómo invocarlas
<code>examples.md</code>	Si el agente comete errores de formato recurrentes	Proporciona ejemplos de respuestas correctas/incorrectas, patrones de código y plantillas de respuesta

<code>workflows.md</code>	Si haces tareas repetitivas (deploy, migraciones, PRs)	Documenta pasos exactos para procesos comunes. El agente los sigue literalmente
---------------------------	--	---

Ejemplo rápido de `tools.md`

markdown

```
# Herramientas permitidas
- `npm run lint` → Validar estilo de código
- `npm run test -- --coverage` → Ejecutar tests y ver cobertura
- `npx prisma migrate dev` → Aplicar migraciones en desarrollo
- `git diff --staged` → Revisar cambios antes de commit
- `curl -s http://localhost:3000/api/health` → Verificar servidor

# Herramientas prohibidas
- `rm -rf` sin confirmación
- Modificar `.env` directamente
- Ejecutar comandos de producción sin aprobación
```

✓ Checklist de implementación

- Renombrar `systemprompt.md` → `.ai/system.md`
- Crear `rules.md`, `context.md`, `agents.md` con contenido alineado
- Validar que no haya contradicciones entre archivos
- Ejecutar: `opencode "Resume tus 3 reglas más importantes"` para verificar carga
- Añadir `tools.md` si tu agente ejecuta comandos
- Commit inicial: `chore(ai): initialize .ai/ folder with standard OpenCode config`